16.1.1.2 Splitting entities

The `Post` entity can be split into several sub-entities according to the three distinct set of attributes:

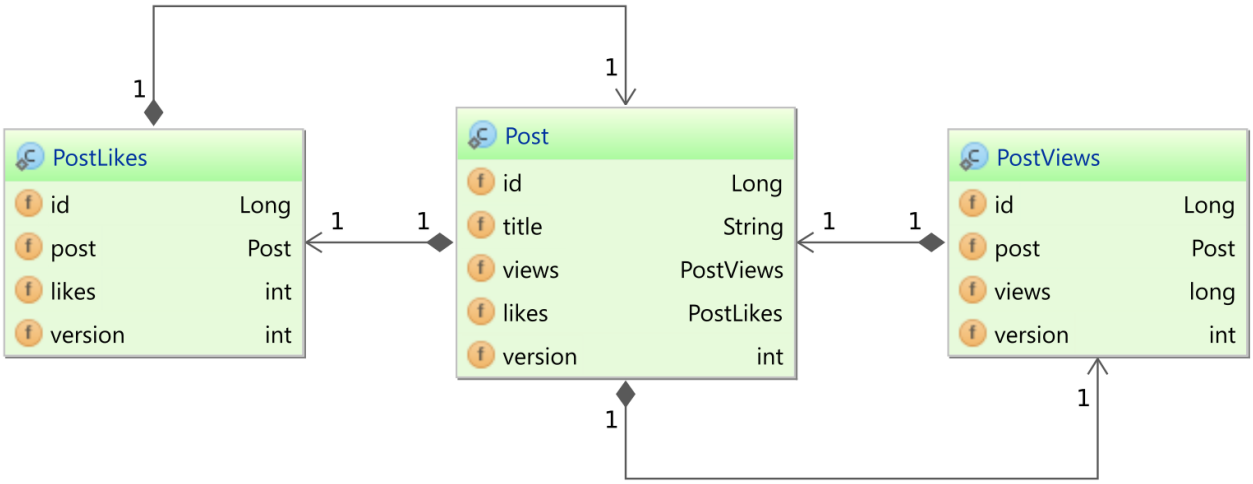

Figure 16.3: Post entity version split

While the `title` attribute remains in the `Post` parent entity, the `likes`, and the `views` attributes are moved to distinct entities. The `PostLikes` and `PostViews` entities are associated with the `Post` parent entity in a bidirectional one-to-one relationship. The `PostLikes` and `PostViews` entity identifiers are also foreign keys to the `post` table primary key.

Each entity has its own `version` attribute. Whenever the `Post` `title` is changed, it is only the `Post` entity `version` that is checked and incremented. When the `views` attribute is updated, only the `PostViews` entity is going to be affected. The same is true for incrementing `likes` which are stored in the `PostLikes` entity.

While breaking a larger entity into several sub-entities can help address optimistic locking conflicts, this strategy has its price. This rather extreme data normalization strategy can have an impact on read operation performance because data is scattered across several tables. If the whole aggregate is needed to be fetched, the data access layer will require to join several tables or execute additional secondary select statements.

The second-level cache can mitigate the read operation performance penalty. Actually, the root entity split can improve the second-level cache performance, especially for read-through strategies (e.g. `NONSTRICT_READ_WRITE`). If the `views` attribute is modified, only the `PostViews` cache entry needs to be invalidated, whereas the `Post` and the `PostLikes` remain unaffected.

When running the previous exercise, there is no longer any conflict being generated:

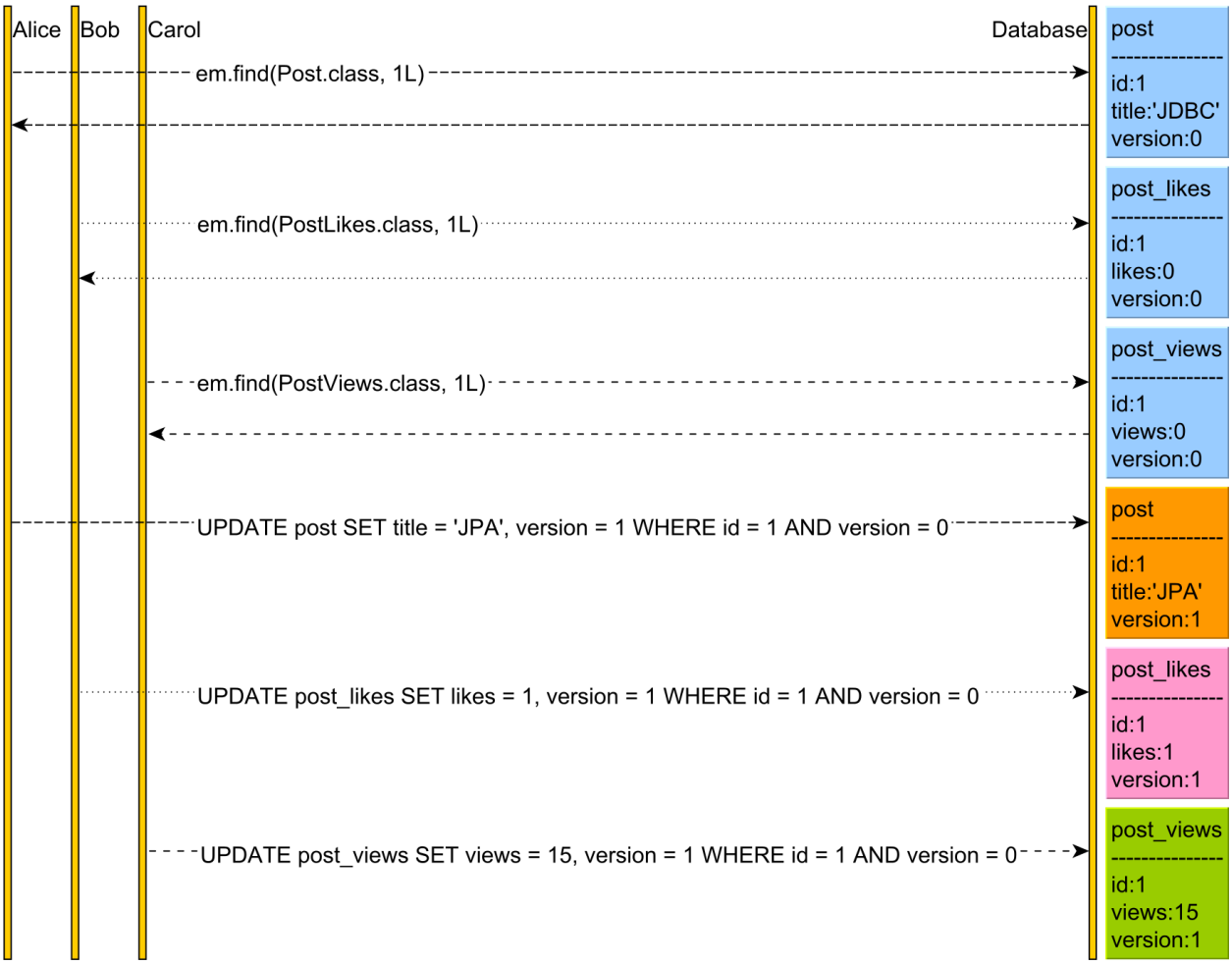

Figure 16.4: Optimistic locking multiple versions

Lost updates are prevented at the entity-level only so the three distinct attributes can be updated concurrently without generating any conflict. However, conflicts can still occur if the same attribute is getting updated by two concurrent transactions.

Designing a Domain Model must take into consideration both read and write data access patterns. Splitting entities by write responsibility can reduce optimistic locking false positives when the write ratio is relatively high.

16.1.1.3 Versionless optimistic locking

Although having a numerical version attribute is the most common optimistic concurrency control strategy, Hibernate offers a versionless optimistic locking mechanism which is not supported by JPA 2.1 specification. To switch to the versionless optimistic locking mechanism, the `@OptimisticLocking` annotation must be configured at the entity level.

The `org.hibernate.annotations.OptimisticLocking` annotation comes with a `type` attribute that can take the following values:

- NONE - The implicit optimistic locking mechanism is disabled.
- VERSION - The implicit optimistic locking mechanism uses a numerical or a timestamp version attribute.
- DIRTY - The implicit optimistic locking mechanism uses only the attributes that have been modified in the currently running Persistence Context.
- ALL - The implicit optimistic locking mechanism uses all entity attributes.

`OptimisticLockType.ALL` and `OptimisticLockType.DIRTY` also require the `@DynamicUpdate` annotation because update statements must be rewritten so that either all attributes or the ones that were modified are included in the where clause criteria. For these two optimistic lock types, the entity versioning mechanism is based on the hydrated state snapshot that was stored when the entity was first loaded in the currently running Persistence Context.

To see how the `OptimisticLockType.ALL` option works, consider the following `Post` entity mapping:

```
@Entity @Table(name = "post") @DynamicUpdate
@OptimisticLocking(type = OptimisticLockType.ALL)
public class Post {

    @Id
    private Long id;

    private String title;

    //Getters and setters omitted for brevity
}
```

When using the aforementioned `Post` entity to rerun the example defined at the beginning of the Hibernate implicit optimistic locking section, Hibernate generates the following output:

```
INSERT INTO post (title, version, id)
VALUES ('High-Performance Java Persistence', 0, 1)

UPDATE post SET title = 'High-Performance JDBC'
WHERE id = 1 AND title = 'High-Performance Java Persistence'
```

If the `Post` entity had more attributes, all of them would be included in the SQL where clause.

> The `OptimisticLockType.ALL` is useful when the underlying database table cannot be altered in order to add a numerical `version` column. Because it takes into consideration all entity attributes, the `OptimisticLockType.ALL` option behaves just like a single global version attribute, and write conflicts can occur even if two concurrent transactions are modifying non-overlapping attribute sets.

Even if the entity splitting method can address the non-overlapping attribute sets conflict, too much data normalization can affect read operation performance. The `OptimisticLockType.DIRTY` option can deal with this issue, and so lost updates are prevented for the currently modified attributes.

To demonstrate it, the following `Post` entity mapping is going to be used while running the same test case employed in the resolve optimistic locking conflicts section:

```
@Entity @Table(name = "post") @DynamicUpdate
@OptimisticLocking(type = OptimisticLockType.DIRTY)
public class Post {

    @Id
    private Long id;

    private String title;

    private long views;

    private int likes;

    //Getters and setters omitted for brevity
}
```

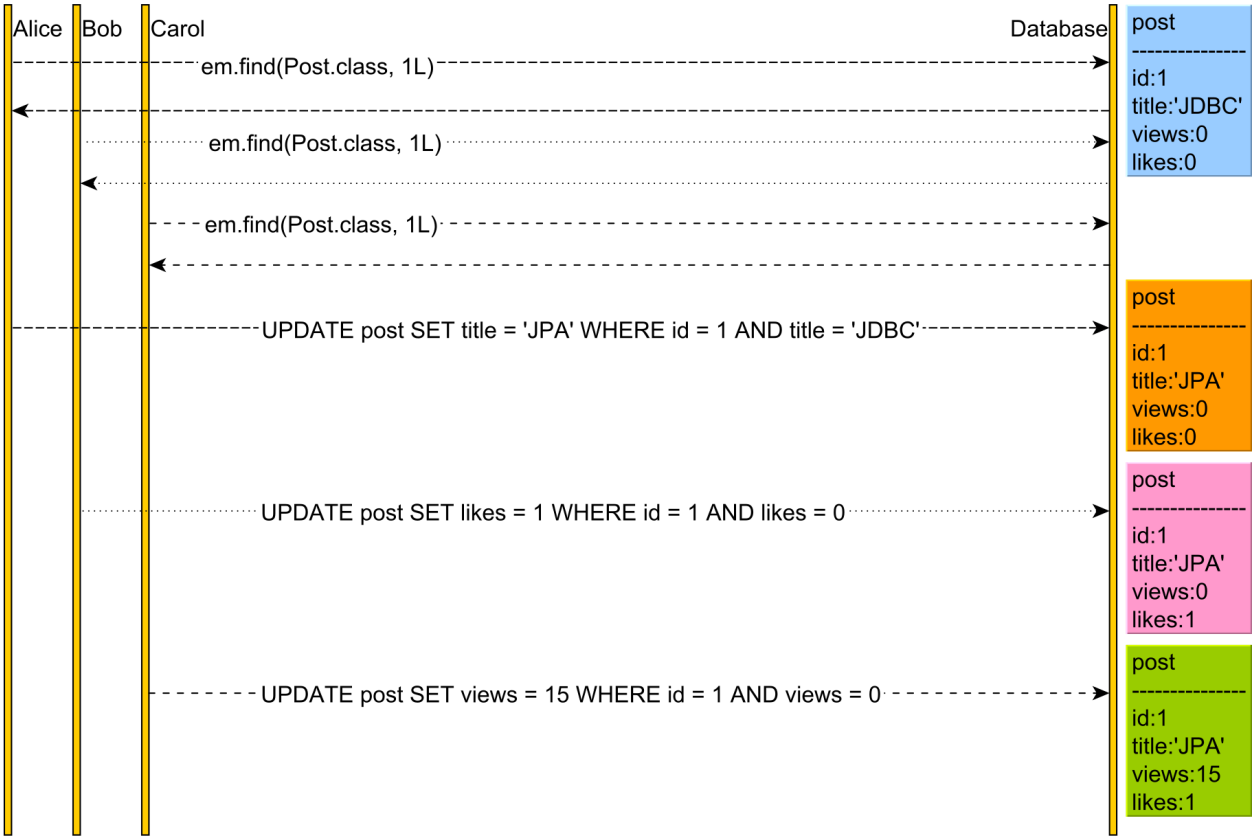

Figure 16.5: Optimistic locking dirty attributes

The `OptimisticLockType.DIRTY` option allows concurrent users to update distinct attributes without causing any conflict. However, conflicts can still occur when two concurrent transactions are updating the same attribute. Therefore, lost updates are prevented on a per-attribute basis.

For heavy-write data access layers, it is not uncommon to split an entity into multiple parts, each individual subentity containing attributes that need to be updated atomically. If, from a writing perspective, attributes are independent, then the `OptimisticLockType.DIRTY` mechanism is also a viable alternative.

Preventing lost updates is essential for data integrity, but the prospect of having transactions aborted due to non-overlapping attribute changes is undesirable. To cope with this issue, entities need to be carefully modeled based on both read and write data access patterns.